



React Routing

1. Need and installation
2. Different components of routing
3. Adding routes
4. Dynamic routing

- Routers help create and navigate between the different URLs that make up your web application.
- They allow the user to move between the components of your app
- React Routing library
 - React Router is a popular routing library for React applications.
 - It manages navigation by assigning component to specific URL
 - It allows programmatic routing as well as sharing data on URL etc

- Single Page Application
 - In single page apps, there is only single html page. We are reusing the same html page to render the different components based on the navigation.
 - If any server side calls are required that are always made asynchronously.
- Installation
 - React routing library can be installed using npm by using the command as `npm install react-router-dom`
 - `react-router-dom` is the variation of core library for web applications

- There are three primary categories of components
- Routers, like `<BrowserRouter>` and `<HashRouter>`
 - For web projects, react-router-dom provides `<BrowserRouter>` and `<HashRouter>` routers.
 - A `<BrowserRouter>` uses regular URL paths and a `<HashRouter>` stores the current location in the hash portion of the URL
- Route matchers, like `<Route>` and `<Switch>`
 - There are two route matching components: `Routes` and `Route`.
 - Whenever URL(location) changes, `<Routes>` look for matching `<Route>`
 - More specific (typically longer) paths should be written before less-specific ones
- Navigation like `<Link>`
 - For `<Link>` element, anchor tag gets rendered
 - It creates a link for navigating to other component

- Using BrowserRouter
 - Wrap `<App>` element in `<BrowserRouter>`
- Creating Links and matching Routes
 - Link element defines the URL path which gets created after clicking the link.
 - For the specified path in Link, a Route needs to be defined using path and element attributes
 - Example
`<Link to="/about">About</Link>`
Matching route
`<Route path="/about" element={ <About /> } />`

4. Dynamic routing

- Accessing data on routes
- There are two primary methods to send data along with route

	Path parameters	Query parameters
Navigation	<code>navigate('/users/123')</code>	<code>navigate('/users?id=123')</code>
Route definition	<code><Route path='/users/:uid' element={<UserProfile/>} /></code>	<code><Route path='/users' element={<UserProfile/>} /></code>
Accessing data in component	<code>const { uid } = useParams();</code>	<code>const { search } = useLocation(); const queryParams = new URLSearchParams(search); const query = queryParams.get('id');</code>